

Entrega 1 · Inventario de Minecraft

1. Objetivo de la práctica

Desarrollar una aplicación en JavaScript ES6 que simule el inventario de un jugador de Minecraft. La práctica servirá para integrar los contenidos trabajados en la unidad: variables, tipos de datos, operadores, estructuras de control, arrays, matrices, funciones, arrow functions, clases, objetos, getters y setters.

Importante: en esta entrega no se utilizará todavía el DOM. La interacción se realizará mediante `prompt()`, `confirm()`, `alert()` y `console.log()`.

2. Planteamiento

El inventario se representará mediante una matriz de objetos. Cada posición de la matriz será un hueco del inventario y podrá contener un objeto de tipo `Item` o estar vacía (`null`).

La primera fila de la matriz representará la barra de accesos rápidos. El resto de filas representarán el inventario principal.

Item 0,0	Item 0,1	Item 0,2	Item 0,3	Item 0,4	Item 0,5	Item 0,6	Item 0,7	Item 0,8
Item 1,0	Item 1,1	Item 1,2	Item 1,3	Item 1,4	Item 1,5	Item 1,6	Item 1,7	Item 1,8
Item 2,0	Item 2,1	Item 2,2	Item 2,3	Item 2,4	Item 2,5	Item 2,6	Item 2,7	Item 2,8
Item 3,0	Item 3,1	Item 3,2	Item 3,3	Item 3,4	Item 3,5	Item 3,6	Item 3,7	Item 3,8

3. Clase Item

Cada objeto almacenado en el inventario será una instancia de la clase `Item`. La clase deberá contener las siguientes propiedades:

Propiedad	Descripción
<code>nombre</code>	Nombre del objeto.
<code>descripcion</code>	Breve descripción del objeto.
<code>cantidad</code>	Número de unidades almacenadas en ese hueco.
<code>maxStack</code>	Número máximo de unidades que puede contener un stack de ese objeto.

La clase deberá incluir, como mínimo:

- Un constructor que inicialice todas las propiedades.
- Un getter y un setter para cantidad.
- El setter de cantidad deberá impedir valores negativos y valores superiores a maxStack.
- Un método mostrarInformacion() que devuelva una cadena con los datos principales del objeto.

```
class Item {  
    constructor(name, description, quantity, maxStack) {  
        // Completar  
    }  
  
    set quantity(value) {  
        // Validar cantidad  
    }  
  
    get quantity() {  
        // Devolver cantidad  
    }  
  
    showInfo() {  
        // Completar  
    }  
}
```

4. Creación del inventario

Crea una matriz de 4 filas y 9 columnas. Las posiciones vacías deberán contener null. Puedes inicializar algunos huecos con objetos para facilitar las primeras pruebas.

- Piedra: stack máximo 64.
- Manzana: stack máximo 64.
- Antorcha: stack máximo 64.
- Espada de diamante: stack máximo 1.
- Pico de hierro: stack máximo 1.

5. Menú principal

La aplicación deberá mostrar un menú repetidamente hasta que el usuario elija salir. Se recomienda utilizar un bucle do...while.

```
INVENTARIO DE MINECRAFT  
=====  
1. Mostrar inventario completo  
2. Mostrar barra de accesos rápidos  
3. Buscar un objeto  
4. Añadir un objeto al inventario
```

```
5. Mover un objeto
6. Eliminar un objeto
7. Mostrar huecos libres
8. Mostrar el objeto más abundante
0. Salir
```

6. Funcionalidades obligatorias

6.1. Mostrar inventario completo

Recorre toda la matriz y muestra el contenido de cada hueco. Los huecos vacíos deberán identificarse como VACÍO. Debe quedar claro qué posición ocupa cada objeto.

6.2. Mostrar barra de accesos rápidos

Muestra únicamente la primera fila del inventario. Debes recorrerla mediante `forEach()` o un bucle equivalente.

6.3. Buscar un objeto

Solicita un nombre y busca el objeto en todas las posiciones. Si existe, muestra su fila, columna, cantidad y descripción. Si aparece en varios stacks, muestra todas sus posiciones.

6.4. Añadir un objeto

Solicita los datos necesarios para crear o añadir un Item. Si ya existe un stack del mismo objeto con espacio disponible, añade unidades sin superar `maxStack`. Si no cabe todo, utiliza otro hueco libre. Si no hay espacio suficiente, informa al usuario.

6.5. Mover un objeto

Solicita posición de origen y posición de destino. Si el destino está vacío, mueve el objeto. Si contiene el mismo tipo de objeto y el stack no está completo, combina cantidades respetando `maxStack`. Valida siempre filas y columnas.

6.6. Eliminar un objeto

Permite vaciar un hueco concreto del inventario asignando `null` a esa posición. Antes de eliminarlo, pide confirmación al usuario.

6.7. Mostrar huecos libres

Cuenta y muestra el número de posiciones `null` existentes en todo el inventario. Muestra también cuántos huecos libres quedan en la barra rápida.

6.8. Mostrar el objeto más abundante

Calcula qué objeto tiene mayor cantidad total sumando todos sus stacks. Si un objeto aparece en varias posiciones, sus cantidades deben sumarse.

7. Organización del código

El programa deberá dividirse en funciones. No se admitirá que toda la lógica se encuentre dentro de un único bloque de código. Como orientación, pueden utilizarse funciones similares a las siguientes:

- `showInventory()`
- `showQuickAccessBar()`
- `searchItem()`
- `addItem()`
- `moveItem()`
- `deleteItem()`
- `countFreeSpaces()`
- `getMostQuantityStack()`

8. Validaciones

La aplicación deberá controlar, como mínimo, las siguientes situaciones:

- Las filas y columnas indicadas existen.
- No se pueden introducir cantidades negativas.
- La cantidad de un Item no puede superar su `maxStack`.
- No se puede mover o eliminar desde un hueco vacío.
- No se puede añadir un objeto si el inventario no dispone de espacio suficiente.
- Las opciones de menú incorrectas no deben finalizar el programa.

9. Formato de entrega

La entrega deberá contener una carpeta con la siguiente estructura mínima:

```
Entrega1_NombreYAPELLIDO /  
├─ index.html  
├─ js/  
│   └─ inventario.js
```

10. Criterios de evaluación

CRITERIO	PUNTUACIÓN
CLASE ITEM, CONSTRUCTOR, OBJETOS, GETTER Y SETTER	1,5 puntos
USO CORRECTO DE ARRAYS Y MATRIZ DE OBJETOS	1,5 puntos
ESTRUCTURAS DE CONTROL Y RECORRIDOS	1,0 puntos
FUNCIONES Y ORGANIZACIÓN DEL CÓDIGO	1,5 puntos
FUNCIONALIDADES DEL MENÚ	2,0 puntos
VALIDACIÓN Y TRATAMIENTO DE ERRORES	1,0 puntos
USO DE ES6 / ARROW FUNCTION	0,5 puntos
CLARIDAD, NOMBRES, INDENTACIÓN Y COMENTARIOS	1,0 puntos
TOTAL	10 puntos